

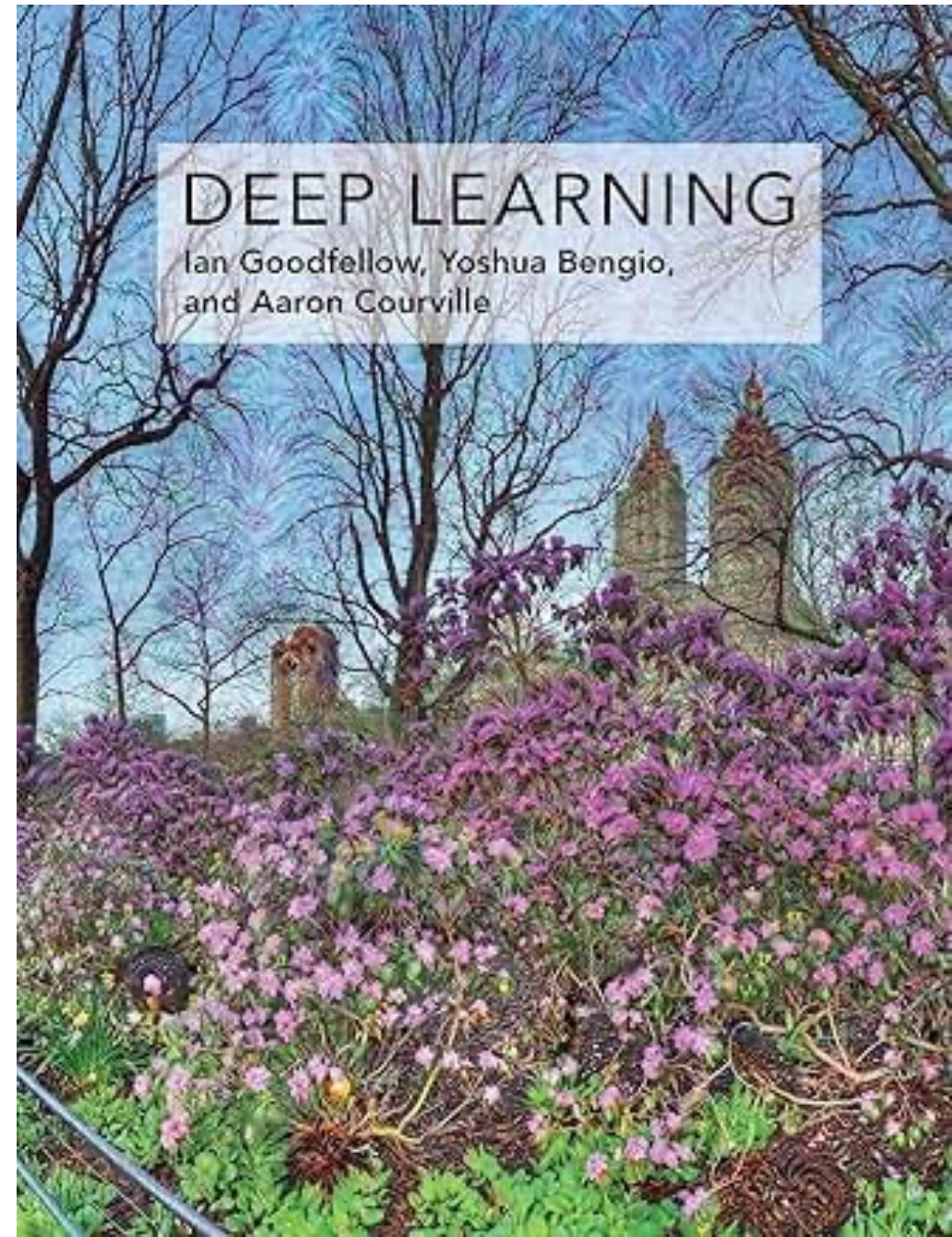


CSCI 6032: Lecture 4-5: Agents

Instructor: Dr. David A. G. Harrison

Things I want to cover today...

- **Agents**
 - Setting up GitHub Copilot CLI
 - What makes a CLI agent different from a chat window
 - Tools, permissions, and the agent loop
 - Skills: packaging repeatable workflows
- **Do it securely.**



DATES OF INTEREST

10/1	Th	Midterm
10/5	M	Progress reports due
10/12	M	Latest drop date

Office Hours

207 Weir Hall

Monday	11:00 AM-12:00 PM
Wednesday	1:00-2:00 PM

HW1 (Reminder)

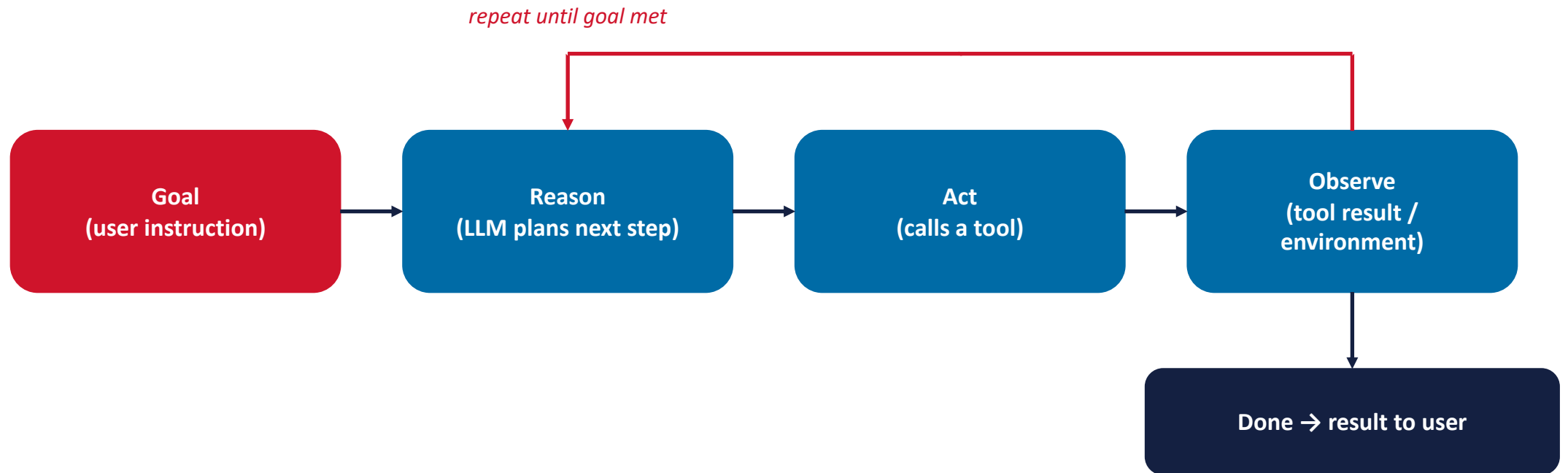
On github already...

Due today at 11pm

Submit as notebook via blackboard.

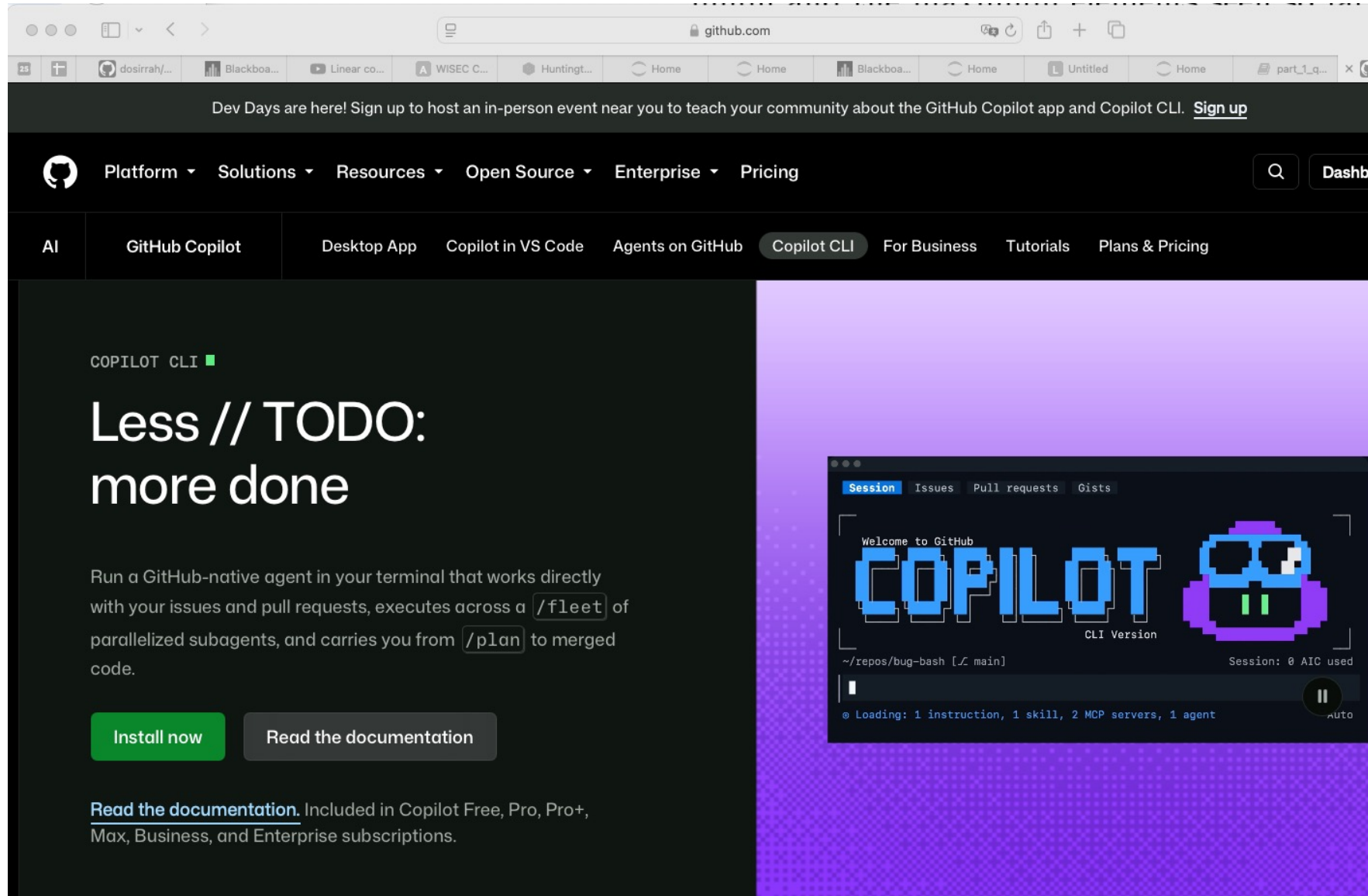
What Is an Agent?

A program that loops: reason about the goal, act in the world, observe what happened — and repeat until the goal is done.



Agent = model (reasoning) + tools (ability to act) + a loop, operating with some autonomy toward a goal.

Agents: GitHub Copilot



Dev Days are here! Sign up to host an in-person event near you to teach your community about the GitHub Copilot app and Copilot CLI. [Sign up](#)

Platform Solutions Resources Open Source Enterprise Pricing

AI GitHub Copilot Desktop App Copilot in VS Code Agents on GitHub Copilot CLI For Business Tutorials Plans & Pricing

COPILOT CLI

Less // TODO: more done

Run a GitHub-native agent in your terminal that works directly with your issues and pull requests, executes across a `/fleet` of parallelized subagents, and carries you from `/plan` to merged code.

[Install now](#) [Read the documentation](#)

[Read the documentation.](#) Included in Copilot Free, Pro, Pro+, Max, Business, and Enterprise subscriptions.



Ole Miss

Agents: GitHub Copilot

<https://github.com/settings/education/benefits>



David Harrison (dosirrah)

Your personal account

 Menu

 Switch context ▾

**Free Student
edition.**

GitHub Education

Free GitHub developer resources for students and teachers

Get Copilot for free, 180 monthly Codespaces hours for cloud coding, unlimited private repositories with GitHub Pro or Team, and dozens of premium tools in the Student Developer Pack.

GitHub Copilot

Command-Line Interface

- **Runs in terminal (shell)**
- **Can install in linux, windows, or macOS.**

Use one of these commands:

- **Cross-platform (npm)**

Prerequisite: Node.js 22 or later.

Bash

```
npm install -g @github/copilot
```

- **Windows (WinGet)**

Bash

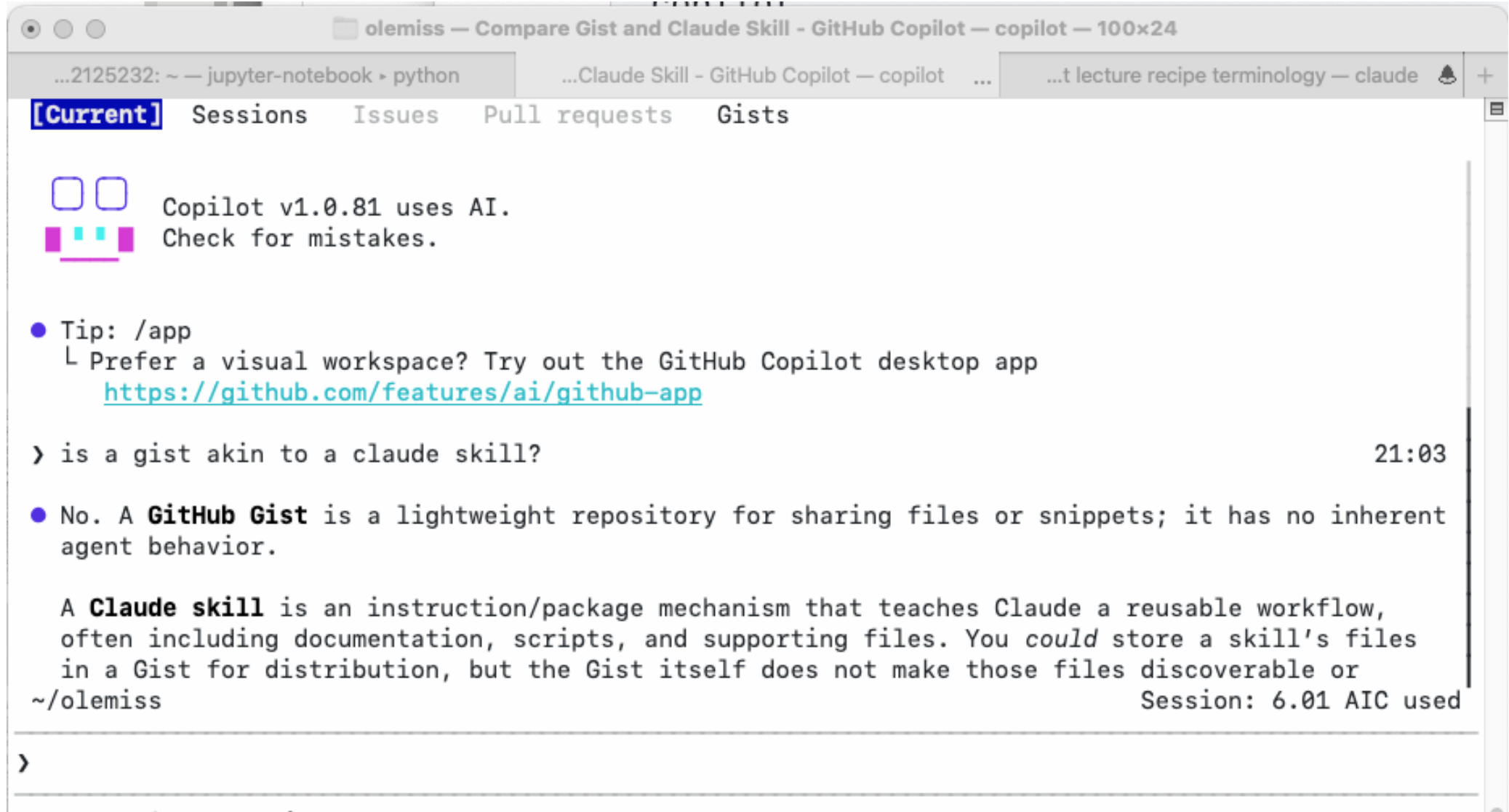
```
winget install GitHub.Copilot
```

- **macOS/Linux (Homebrew)**

Bash

```
brew install --cask copilot-cli
```

GitHub Copilot



```
olemiss — Compare Gist and Claude Skill - GitHub Copilot — copilot — 100x24
...2125232: ~ — jupyter-notebook > python
...Claude Skill - GitHub Copilot — copilot
...t lecture recipe terminology — claude
[Current] Sessions Issues Pull requests Gists

Copilot v1.0.81 uses AI.
Check for mistakes.

• Tip: /app
  L Prefer a visual workspace? Try out the GitHub Copilot desktop app
  https://github.com/features/ai/github-app

> is a gist akin to a claude skill? 21:03

• No. A GitHub Gist is a lightweight repository for sharing files or snippets; it has no inherent agent behavior.

A Claude skill is an instruction/package mechanism that teaches Claude a reusable workflow, often including documentation, scripts, and supporting files. You could store a skill's files in a Gist for distribution, but the Gist itself does not make those files discoverable or
~/olemiss Session: 6.01 AIC used

>
```

What Makes a CLI Agent Different from a Chat Window?

A chat window answers questions; a CLI agent takes action

- Chat: you copy code out, run it, and paste results back in
- CLI agent: reads, edits, and runs code directly in your project

It can take multiple steps on its own

- Plans, calls tools, checks results, and continues — without a relay

It needs permission boundaries

- Because it acts instead of just talking, what it's allowed to do matters

Tools

A tool is a specific capability the agent can invoke

- Read / write / edit files
- Run shell commands
- Search the web, browse a page
- Query an API, a database, a service

The agent decides which tool to call and with what arguments

Tool results feed back into its reasoning for the next step

More tools = more capability, but also more surface area for mistakes

Permissions

Every tool call is a real action — permissions decide which ones need a human first

- **Auto-approved**
 - Safe, reversible actions (e.g., reading a file, listing a directory)
- **Ask first**
 - Risky or hard-to-reverse actions (e.g., editing files, running a command, pushing to git)
- **Denied**
 - Actions explicitly out of bounds for the task

Good permission design assumes the agent will eventually be wrong about something

- You're bounding the blast radius of a mistake, not just approving good behavior

Skills: Packaging Repeatable Workflows

A skill is a saved, reusable recipe for a task you do more than once

- **Instructions the agent follows step by step (e.g., “post a lecture”, “grade a homework”)**
- **Can bundle scripts, templates, and checklists alongside the instructions**

Why bother?

- **Consistency — the same steps happen the same way every time**
- **Less re-explaining — you don't retype the process in every session**
- **Shareable — teammates (or future you) can reuse the same workflow**

Creating a Skill

A skill is just a folder with instructions the agent can discover and load

- A short description of when to use it (its trigger)
- Step-by-step instructions written in plain language
- Optional helper scripts or reference files it can call

The agent decides on its own when a skill applies

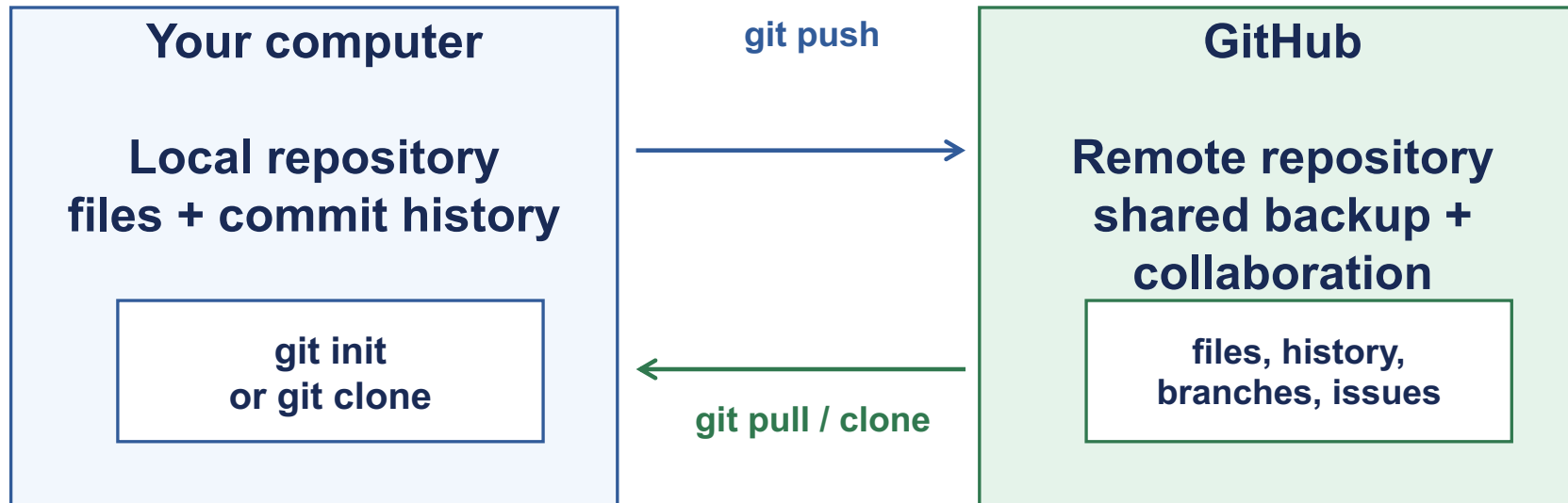
- You describe the task; if it matches a skill's description, the agent invokes it

Good skills are narrow and explicit

- One clear workflow per skill beats one skill that tries to do everything

GitHub Repositories: A Shared Project Home

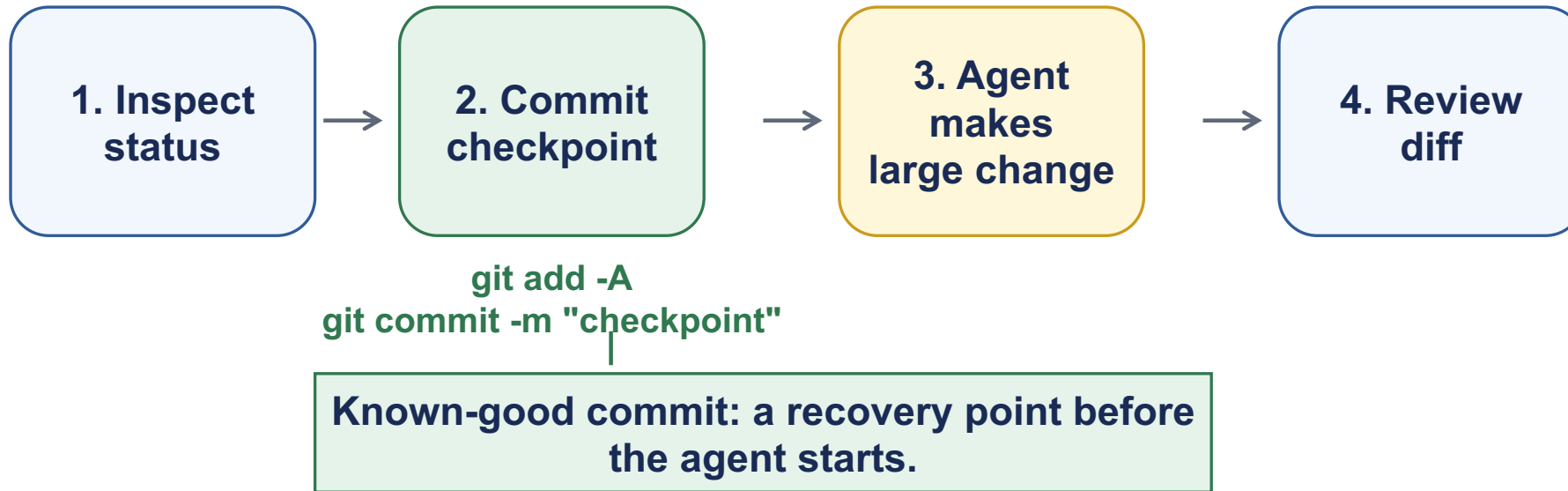
A repository stores project files and their history, locally on your computer and remotely on GitHub.



Git gives you a timeline: each commit is a named snapshot you can compare with or return to later.

Git + Agents: Make a Checkpoint First

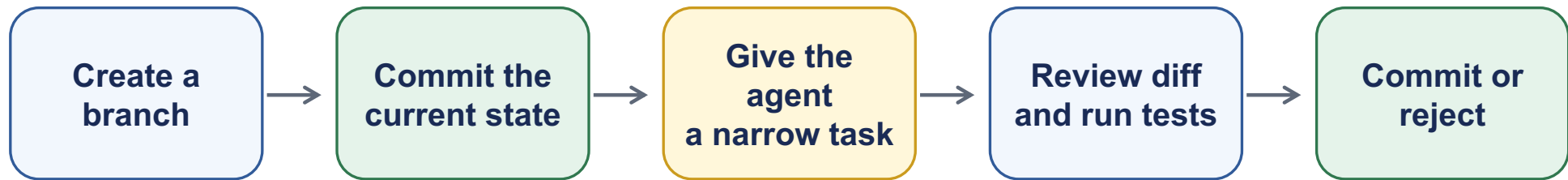
Agents work directly in your checkout. Create a commit before a large refactor or another consequential action.



After the agent finishes, inspect git diff and tests. Commit accepted work; restore only the unwanted files or changes.

A Safe Agent Workflow for Big Changes

Use Git as the control plane around agent work: isolate the task, preserve a checkpoint, and review the result.



If the change is not right, keep the checkpoint and discard only the agent's unaccepted work.

Do not ask an agent to make broad, irreversible changes in your only copy of a project.

Running an Agent Securely

An agent can read files, edit code, and run commands. Treat it like a capable new process on your computer.

Agent on host OS
Can act with your user's access

limit

Containerized agent
Gets only the access you give it

Use a separate working directory.

Do not expose your home directory, SSH keys, or cloud credentials.

Keep permission prompts on; inspect changes before accepting them.

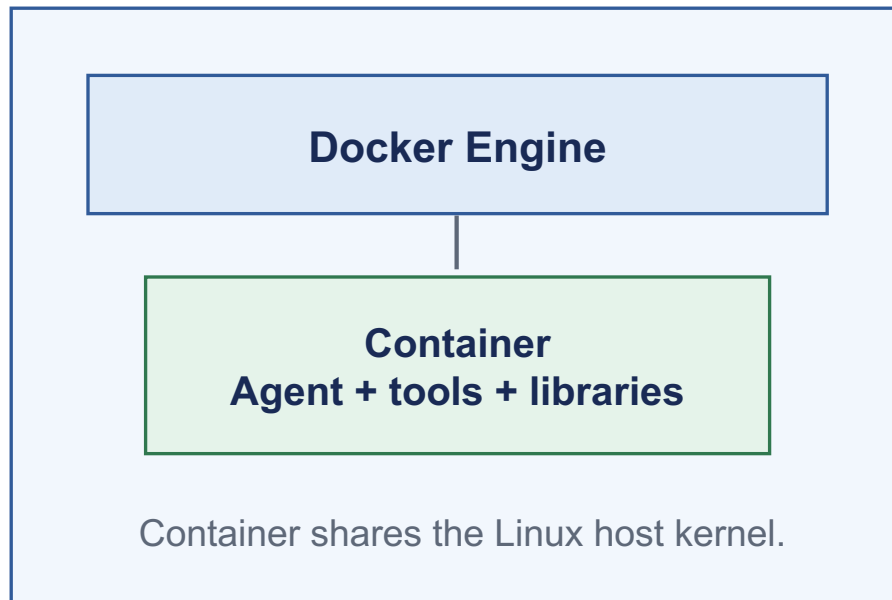
Isolation reduces risk; it does not make untrusted code safe.

Principle of least privilege: give each agent the smallest set of permissions it needs.

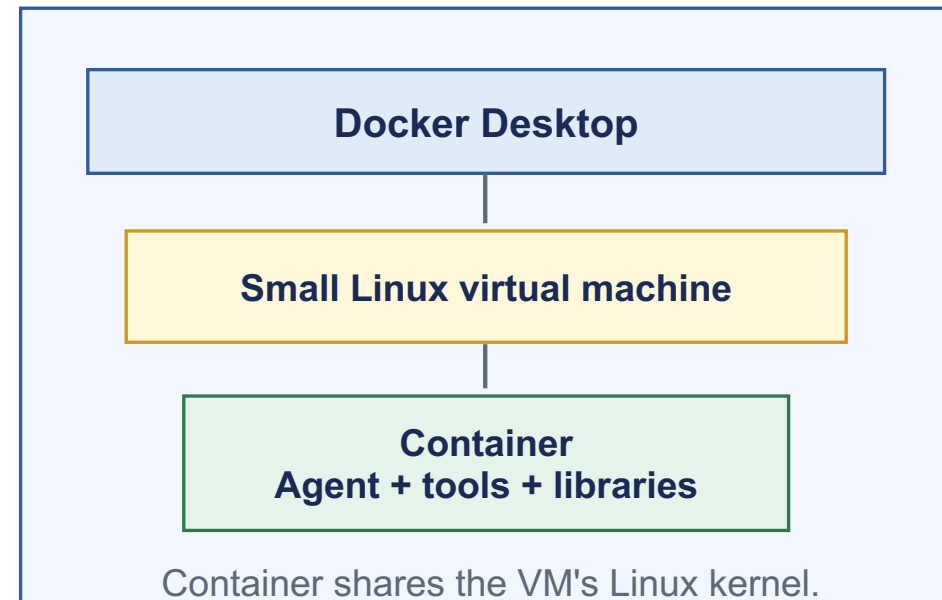
What a Docker Container Is

A container packages an application and its dependencies, while isolating its processes and filesystem view.

When the host is Linux



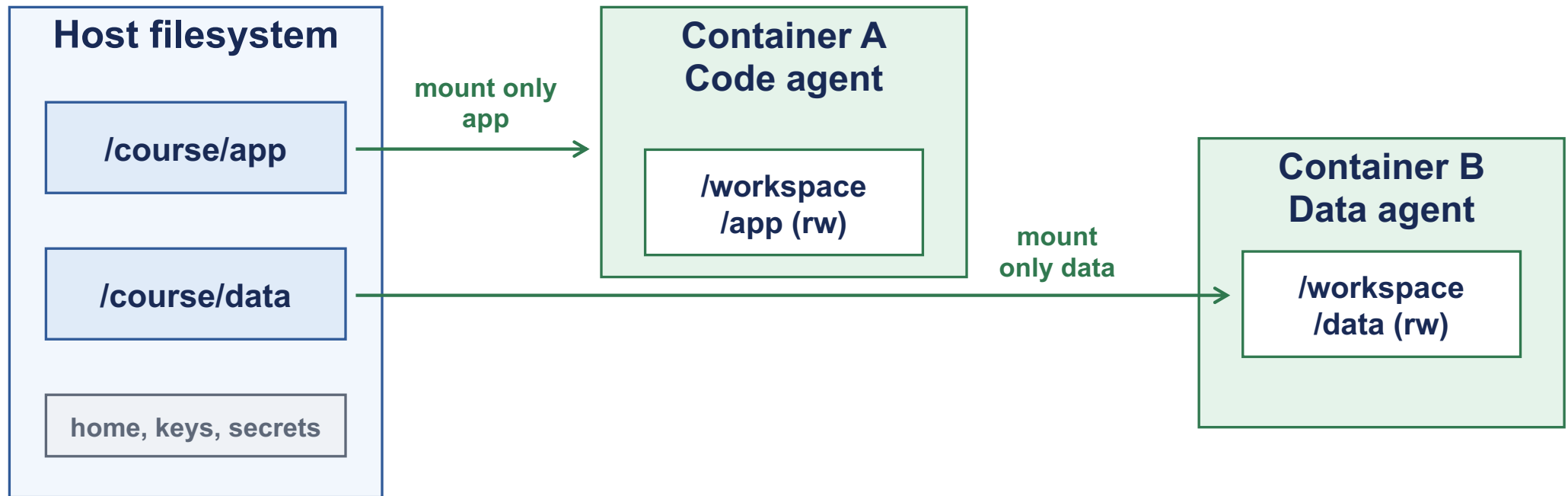
When the host is macOS or Windows



Containers are lighter than full VMs, but they are not a substitute for careful permissions and narrow mounts.

Constrain Each Agent with Scoped Mounts

A bind mount exposes one selected host directory inside a container. Do not mount more than the task requires.



Neither container can see the host paths that were not mounted into it.

A Safer Default for Agent Containers

Start restrictive, then grant only the capability that a specific task needs.

```
docker run --rm \  
  --user 1000:1000 \  
  --read-only --tmpfs /tmp \  
  --cap-drop ALL --network none \  
  -v "$PWD/project":/workspace:rw \  
  -w /workspace agent-image
```

Run as a non-root user.

Make the container filesystem read-only; use a temporary scratch area.

Remove Linux capabilities except as needed.

Disable network access by default then allow access to copilot.

Mount one project directory, not your whole home directory.

Before using an agent: use version control, keep secrets outside the mount, and review commands or file changes at permission boundaries.

MCP: A Standard Way to Give Agents Tools

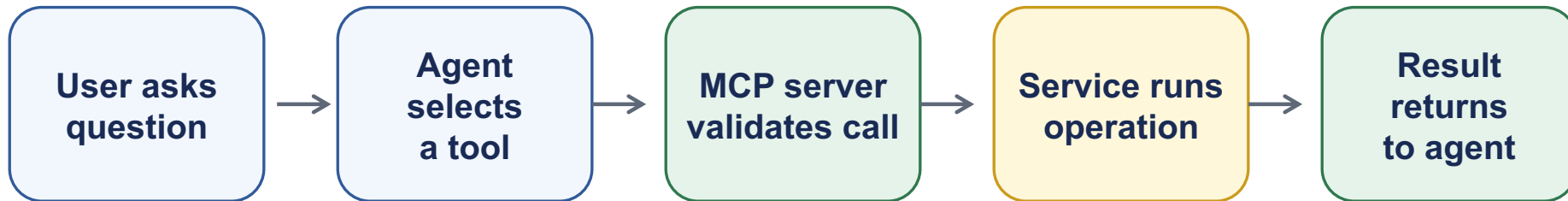
Model Context Protocol (MCP) is a common interface between an AI agent and external tools or data services.



MCP lets an agent discover a tool's name, inputs, and description instead of needing a custom integration for every service.

How an MCP Tool Call Works

The agent proposes a structured call; the MCP server validates it, contacts the service, and returns a structured result.

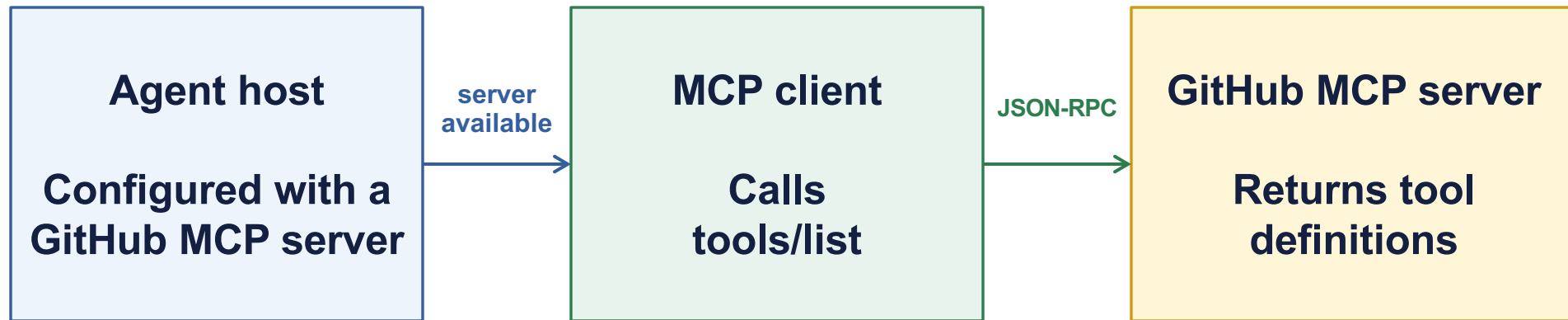


Permissions still apply: a server can offer read-only tools, require confirmation for writes, or deny a request.

MCP expands what an agent can do; it does not remove the need for least privilege and human review.

MCP Setup and Tool Discovery

STEPS 1–3



```
search_issues(query)  get_issue(number)  add_label(number, label)
post_comment(number, text)
```

**The host contains the MCP client.
The model receives tool descriptions through the host.**

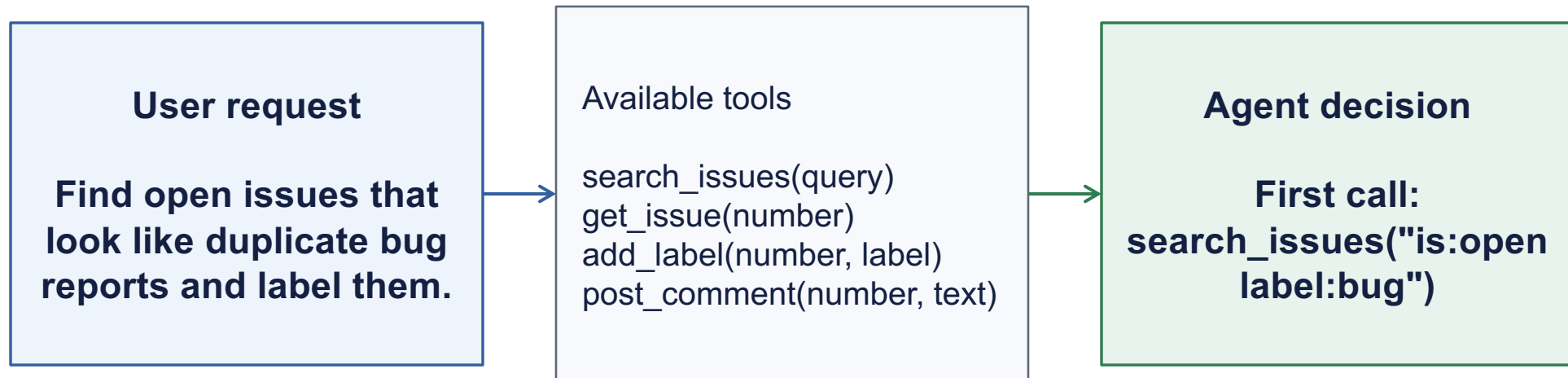
Step 3 return result

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "tools/list"  
}
```

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "result": {  
    "tools": [  
      {  
        "name": "search_issues",  
        "description": "Search GitHub issues",  
        "inputSchema": {  
          "type": "object",  
          "properties": { "query": { "type": "string" } },  
          "required": ["query"]  
        }  
      },  
      {  
        "name": "get_issue",  
        "description": "Retrieve a GitHub issue",  
        "inputSchema": {  
          "type": "object",  
          "properties": { "number": { "type": "integer" } },  
          "required": ["number"]  
        }  
      }  
    ]  
  }  
}
```

Tools Become Available to the Agent

STEPS 4–5



The agent chooses the next operation. MCP provides the mechanism for calling it.

The First Request on the Wire

STEP 6

```
{  
  "jsonrpc": "2.0",  
  "id": 2,  
  "method": "tools/call",  
  "params": {  
    "name": "search_issues",  
    "arguments": {  
      "query": "is:open label:bug"  
    }  
  }  
}
```

What the fields mean

jsonrpc
Message format

id
Matches request and response

method
Standard MCP operation

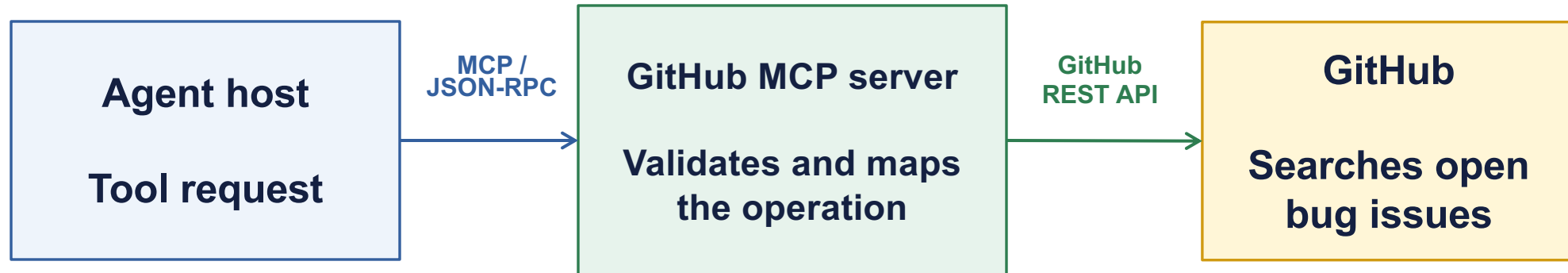
params
Tool name and arguments

The model chooses a tool call. The MCP client sends the protocol message.

The MCP Server Calls GitHub

STEP 7

The server receives tools/call, validates the arguments, and invokes the external service.



MCP standardizes the agent-to-server boundary. The GitHub API performs the underlying operation.

The Result Returns to the Agent

STEPS 8–9

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Found issues #81, #93, #104, and #117."
      }
    ]
  }
}
```

New model context

**Found issues
#81, #93, #104, #117**

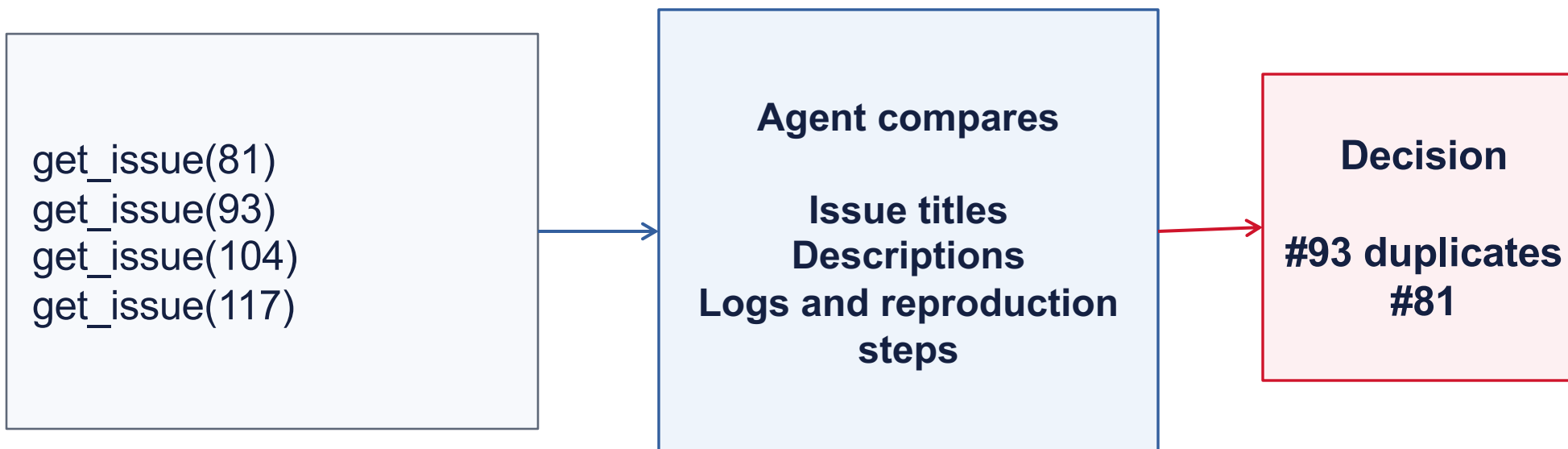
Agent reasons again

**Inspect each issue before
deciding whether any are
duplicates.**

A tool result becomes new context for the model's next decision.

Inspecting Candidate Duplicates

STEP 9 CONTINUED



Each inspection uses another MCP request-result cycle, then the model reasons over the returned data.

The Agent Labels the Duplicate

STEP 10

```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "add_label",
    "arguments": {
      "number": 93,
      "label": "duplicate"
    }
  }
}
```

Who does what

Agent
Decides that a write is needed

MCP
Carries the standardized call
and result

GitHub API
Changes the issue label

Write actions still follow the permissions and confirmation rules configured by the host and server.